PYTHON EXAM

Study & Preparation Guide

Condensed Assessment

Aligned with assessment version 3.0

Practice-first guide
Python fundamentals • Git • PEP 8 & Ruff
OOP & static analysis • pytest

Candidate guide — general syllabus, practice examples, and a four-hour crash plan.
Does not include exam questions or answers.

Contents

How to use this guide 3

Format you should expect 3

Session weights 3

Syllabus by session (with examples) 4

1) Python Fundamentals & Virtual Environments (23 pts) . . . 5

2) Git Basics & Rebase (22 pts) . . . 7

3) PEP 8 & Ruff (18 pts) . . . 8

4) OOP & Static Analysis (23 pts) . . . 10

5) Unit Testing (pytest) (14 pts) . . . 12

Crash plan (~4 hours) 13

Common mistakes (avoid these) 14

How to answer during the exam 14

Final checklist (before you start) 14

Repo resources 15

Note 15

> Candidate guide. Covers the general syllabus, practice examples, and a ~4-hour crash plan. **Does not include exam questions or answers.**

Aligned with assessment version 3.0.

How to use this guide

1. Review the syllabus and examples for each session.
2. Follow the **~4-hour crash plan**.
3. Finish with the final checklist.

Golden rule: practice by writing code and commands. Do not only read.

Format you should expect

Most questions are **scenario-based**, not definition-based. Typical patterns:

Pattern	What you must do
Debug	Given broken code, list every bug and provide the fixed version
Analyze	Given an error message or tool output, explain the cause and fix it
Implement	Write working code that satisfies a stated contract
Refactor	Improve existing code against a standard (PEP 8, SOLID)
Configure	Produce real config (e.g. `pyproject.toml` , CI YAML)

Multi-part questions are common ("provide 1, 2 and 3"). **Answer every numbered part** — partial answers lose points even when the code is right.

Most examples use automotive/IVI context: ECU, CAN bus, sensors, ignition states.

Session weights

Session	Topic	Time	Points
1	Python & Virtual Environments	20 min	23
2	Git Basics & Rebase	20 min	22
3	PEP 8 & Ruff	18 min	18
4	OOP & Static Analysis	20 min	23
5	Unit Testing	15 min	14
Total		**~93 min**	**100**

Sessions 1 and 4 carry the most weight. Do not leave them for last.

Syllabus by session (with examples)

1) Python Fundamentals & Virtual Environments (23 pts)

You should know:

- Debugging aggregation logic (sums/counts, not running averages)
- Dictionary and list comprehensions, including **nested** structures
- Shallow copy vs deep copy **and when to use each**
- Creating a venv with a **specific Python version**
- Why `pip freeze` outside a venv is a problem
- Diagnosing `ImportError` caused by a non-activated venv

Example — aggregation done right:

```python
def average_by_key(pairs: list[tuple]) -> dict:
    sums: dict = {}
    counts: dict = {}

    for key, value in pairs:
        sums[key] = sums.get(key, 0) + value
        counts[key] = counts.get(key, 0) + 1

    return {k: sums[k] / counts[k] for k in sums}
```

Averaging with `(previous + new) / 2` is a classic bug. It only averages the last two values.

Example — nested comprehension:

```python
def group_by_id(records: list[tuple]) -> dict:
    ids = {rec[1] for rec in records}
    return {
        item_id: {
            "timestamps": [ts for ts, rid, _ in records if rid == item_id],
            "values": [val for _, rid, val in records if rid == item_id],
            "count": sum(1 for _, rid, _ in records if rid == item_id),
        }
        for item_id in ids
    }
```

Example — venv with a specific version:

```powershell
py -3.10 -m venv test_env
.\test_env\Scripts\Activate.ps1
pip install -r requirements.txt
python --version
```

```bash
python3.10 -m venv test_env
source test_env/bin/activate
pip install -r requirements.txt
python --version
```

Example — the shallow copy trap:

```python
import copy

base = {"vehicle": "Pick-Up", "settings": {"timeout": 5}}

a = base.copy()                 # nested dict is SHARED
b = copy.deepcopy(base)         # fully independent

a["settings"]["timeout"] = 10   # also changes base
```

Self-check:

- ■ I can build a nested dict with comprehensions
- ■ I can explain and demonstrate shallow vs deep copy with a test-data example

- ■ I can create a venv pinned to a Python version and verify it
- ■ I know why `pip install` before activating causes `ImportError`

2) Git Basics & Rebase (22 pts)

You should know:

- Fixing a wrong remote URL (`git remote set-url`)
- Reading `git branch -vv` , including what `gone` means
- That `-u` and `--set-upstream` are identical
- Removing secrets from **pushed** history (BFG / `filter-branch`)
- Merge vs rebase, including drawing the commit graph
- Resolving a rebase conflict and continuing
- Interactive rebase: `squash` , `reword` , `pick`
- `--force-with-lease` instead of `--force`

Example — wrong remote:

```
git remote set-url origin https://github.com/org/correct-repo.git
git remote -v
git push origin feature/my-branch
```

Example — rebase conflict:

```
git rebase origin/development
# edit conflicted files, keep BOTH changes when both are needed
git add tests/test_vehicle.py
git rebase --continue
git push --force-with-lease
```

`git rebase --skip` **discards your commit entirely**. Know the difference.

Example — interactive rebase:

```
git rebase -i HEAD~6
# in the editor:
# pick   f1g2h3i  Initial structure
# reword e9f0g1h  Update test config
# pick   d7e8f9g  Add integration tests
# squash c5d6e7f  Fix formatting
# squash b3c4d5e  Update tests again
# squash a1b2c3d  WIP - fix typo
```

Key ideas:

- **Merge:** creates a merge commit, preserves history
- **Rebase:** replays commits, rewrites history, no merge commit
- Deleting a secret in a new commit is **not enough** — it stays in history and the key must be rotated

Self-check:

- ■ I can draw the commit graph for merge vs rebase
- ■ I can explain `gone` and restore tracking
- ■ I can purge a file from history and prevent it via `.gitignore` + pre-commit
- ■ I can squash and reword in one interactive rebase

3) PEP 8 & Ruff (18 pts)

You should know:

- Spotting **8+ violations** in one snippet
- Refactoring a multi-parameter function fully to PEP 8
- Arguing whether a violation is justified (not just how to silence it)
- Ruff: `--fix` , `--diff` (dry run), scoping to a directory
- Writing Ruff config in `pyproject.toml`
- Ruff in CI: check-only, no auto-fix

Violations to look for: PascalCase functions, wildcard imports, multiple imports on one line, semicolons, missing spaces around `=` and after commas, spaces inside parentheses, statements on the same line as `if` , `== True` , `%` formatting, single-letter names.

Example — before / after:

```python
# Before
def CalculateTotal(a,b):return a+b

# After
def calculate_total(a: int, b: int) -> int:
    return a + b
```

Ruff commands:

```bash
ruff check .
ruff check . --fix
ruff check . --diff        # show what would change
ruff check tests/
```

Example — `pyproject.toml` :

```toml
[tool.ruff]
line-length = 120
exclude = ["vendor", "build"]

[tool.ruff.lint]
select = ["F", "E", "W", "I", "PT"]

[tool.ruff.lint.per-file-ignores]
"tests/*" = ["E501", "F841"]
```

Example — CI that fails cleanly:

```yaml
- name: Lint with Ruff
  run: |
    pip install ruff
    ruff check . --output-format=github
```

> Never run `--fix` in CI: it mutates files and breaks `git diff --exit-code` . Fix locally via pre-commit.

Justified vs not:

- Long data literal → reformat across lines; length alone is not a justification
- `X` for a matrix in linear algebra → defensible by domain convention
- `lambda` assigned to a name → use `def` (E731)

Self-check:

- ■ I can list 8+ violations without a linter
- ■ I can write a `pyproject.toml` with per-file ignores
- ■ I can explain why `--select ALL` is a bad default

- ■ I know `--diff` for dry runs

4) OOP & Static Analysis (23 pts)

You should know:

- Debugging a class: missing `self` in signatures, unqualified attributes
- Building a hierarchy with `super()` and real polymorphism
- Why Python has no traditional overloading (and what replaces it)
- Refactoring against **SRP, OCP, DIP**
- What `mypy` can and cannot catch before runtime
- Adding type hints (`Optional` , `list[dict]`) to untyped functions

Example — common class bugs:

```python
class ECUSimulator:
    def __init__(self, ecu_id: str, protocol: str) -> None:
        self.ecu_id = ecu_id
        self.protocol = protocol
        self.state = "disconnected"
        self.message_count = 0

    def connect(self) -> None:          # 'self' was missing
        self.state = "connected"        # was a local variable
        print(f"ECU {self.ecu_id} via {self.protocol}")

    def send_message(self, message: str) -> str:
        if self.state != "connected":
            return "Error: Not connected"
        self.message_count += 1         # was missing 'self.'
        return f"Sent: {message}"
```

Example — polymorphism with `super()` :

```python
class TestFramework:
    def __init__(self) -> None:
        self.results: list = []

    def setup(self) -> None:
        self.results.clear()

    def execute(self, test_case: str) -> str:
        raise NotImplementedError


class BDDFramework(TestFramework):
    def execute(self, test_case: str) -> str:
        return f"Gherkin: {test_case}"
```

> Python has no overloading because a later definition replaces the earlier one. Use default arguments, `*args` , or `functools.singledispatch` .

SOLID smells to recognize:

- `if type == "A" ... elif type == "B"` → OCP violation, use subclasses/strategy
- One class that runs tests, saves to DB and sends email → SRP violation
- Class constructing its own dependencies → DIP violation, inject them

Static analysis reality:

- Catches: type mismatches, wrong return types, undefined names (`responce`)
- Misses: logic errors, runtime data issues, race conditions

Self-check:

- ■ I can spot missing `self` bugs instantly
- ■ I can implement a base class + two overrides using `super()`

- ■ I can name the violated SOLID principle and refactor it
- ■ I can add `Optional[str]` / `list[dict]` hints correctly

5) Unit Testing (`pytest`) (14 pts)

You should know:

- Fixing a function **and** its test (validation + `pytest.raises`)
- Mocking hardware interfaces so tests need no device
- Asserting mock calls with exact arguments
- Verifying logging calls
- `@pytest.mark.parametrize` for valid and invalid cases
- Fixtures that reset state between tests

Example — testing errors properly:

```python
import pytest


def calculate_efficiency(distance_km: float, fuel_liters: float) -> float:
    if fuel_liters <= 0:
        raise ValueError("fuel_liters must be positive")
    return distance_km / fuel_liters


def test_raises_on_zero_fuel():
    with pytest.raises(ValueError):
        calculate_efficiency(100, 0)
```

Example — mocking hardware:

```python
import pytest
from unittest.mock import patch


@pytest.fixture
def mock_bus():
    with patch("mymodule.can.interface.Bus") as bus_cls:
        yield bus_cls.return_value


def test_send_message_calls_bus(mock_bus):
    iface = CANInterface("vcan0", 500000)
    assert iface.send_message(0x123, b"\x01") is True
    mock_bus.send.assert_called_once()
```

Example — parametrize both paths:

```python
import pytest


@pytest.mark.parametrize(
    "start,target,expected",
    [
        ("OFF", "ACC", True),
        ("OFF", "START", False),
        ("ON", "START", True),
        ("OFF", None, False),
    ],
)
def test_transitions(start, target, expected):
    machine = IgnitionStateMachine()
    machine.state = start
    assert machine.transition(target) is expected
```

Self-check:

- ■ I can use `pytest.raises` for expected exceptions
- ■ I can mock a hardware dependency with `patch`
- ■ I can assert a mock was called with specific arguments

- ■ I can parametrize valid, invalid and edge cases

Crash plan (~4 hours)

Use this plan if you have **one 4-hour block** the same day or the day before. Time is weighted toward the heaviest sessions.

Block	Time	Topic	What to do
0	10 min	Setup	Open editor, terminal, Python, and Git. Create a `practice/` folder
1	55 min	Python + venv	Fix a broken averaging function. Build a nested comprehension. Create a version-pinned venv. Reproduce the shallow-copy bug
2	45 min	Git	Fix a remote URL. Force a rebase conflict and resolve it. Run one interactive rebase with squash + reword
3	30 min	PEP 8 + Ruff	Find 8+ violations by hand. Write a `pyproject.toml` with per-file ignores
4	55 min	OOP + typing	Debug a class missing `self` . Build a base class + 2 subclasses. Refactor an `if type ==` chain
5	35 min	Testing	Write `pytest.raises` , one mock with `patch` , one parametrize with invalid cases
6	10 min	Review	Complete the final checklist and redo whatever was not fluent

Suggested clock:

- 00:00–00:10 Setup
- 00:10–01:05 Python + venv
- 01:05–01:50 Git
- 01:50–02:20 PEP 8 + Ruff
- 02:20–03:15 OOP + typing
- 03:15–03:50 Testing
- 03:50–04:00 Review

What NOT to do in 4 hours:

- Do not read long docs end to end
- Do not install 10 new tools
- Do not memorize commands without practicing them
- If you get stuck for more than 8 minutes, note the question and move on

Must-do drills:

1. Aggregate values by key with correct averaging (sum/count)
2. Build a nested dict via comprehension
3. Rebase with a conflict, then `--force-with-lease`
4. Fix a class with missing `self` and unqualified attributes
5. Parametrized test that covers an invalid case

Common mistakes (avoid these)

- **Answering only part of a multi-part question** — the most frequent point loss
- Averaging with `(prev + new) / 2` instead of sum/count
- Explaining shallow vs deep copy without saying when to use each
- Running `pip install` before activating the venv
- Confusing virtual environments with Git branches
- `git add branch-name` instead of `git checkout -b`
- Assuming deleting a secret in a new commit removes it from history
- `git push --force` instead of `--force-with-lease`
- Running `ruff --fix` inside CI
- Forgetting `self.` on instance attributes
- Tests with no `assert` , or missing `pytest.raises` for error paths
- Leaving `pass` / TODO in the submission

How to answer during the exam

1. Read the full question and **count the numbered parts**.
2. Answer each part explicitly — label them 1, 2, 3.
3. For debug questions, **list the bugs first**, then give the fixed code.
4. For code: clarity over cleverness.
5. If you cannot recall exact syntax, write the reasoning and the flow.
6. Keep 5–10 minutes at the end to check names, indentation, and `self` .

Suggested answer format:

- Bugs/violations: a numbered list
- Code: a clean `python` , `bash` or `toml` block
- Explanation: 3–6 bullets, not an essay

Final checklist (before you start)

- ■ I can debug aggregation logic and write nested comprehensions
- ■ I can create a venv pinned to a version and explain `ImportError`
- ■ I can demonstrate the shallow-copy trap and fix it
- ■ I can rebase with conflicts and force-push safely
- ■ I can purge a secret from git history
- ■ I can find 8+ PEP 8 violations and write Ruff config

- ■ I can fix `self` -related class bugs
- ■ I can name and repair SOLID violations
- ■ I can mock hardware and parametrize invalid cases
- ■ I answer every numbered sub-question

Repo resources

- `docs/quick-reference.md`
- `docs/usage-guide.md`
- `docs/installation-guide.md`
- `scripts/python/README.md`
- `templates/python-template.py`
- `templates/test_condensed_version/candidate_templates/` (answer sheets)

Note

This guide covers the expected syllabus, not the exam. The assessment rewards **practice, clarity, and technical judgment**, not word-for-word memorization.